Uploading data

Getting a search-engine output into the tool, and what it has to look like.

AT A GLANCE

Where	Data → Upload
Accepts	.csv, .tsv, .txt, .xlsx, .parquet. Anything else is refused by name before it is read.
Levels	Protein, phospho, phosphoprotein and peptide are uploaded separately, one file each. A phosphoprotein matrix can also be built here from an uploaded phospho file rather than supplied — see *Phosphosites to phosphoproteins*.
Shape	Protein, phospho and phosphoprotein are wide: one row per feature, one column per sample. Peptide is the long report as the search engine wrote it.
Does not	Search, normalise, filter, impute or decide which columns are samples. That is the annotation's work.
Replaces	Uploading a level again discards the annotation and the uploaded metadata of that level.

1 What you can upload

The tool reads a table, not a raw file. What is uploaded is the output of a search engine — a protein-group report, a collapsed phosphosite report, a precursor or peptide report — exported to one of five file types. The extension decides how it is parsed, and nothing else does:

- .csv is read as comma-separated text.
- .tsv and .txt are read as tab-separated text. A .txt file that is really comma-separated arrives as one column and is refused for that reason.
- .xlsx is read as a workbook, first worksheet only.
- .parquet is read in batches through its own schema.

Three of the four dataset types are wide matrices: one row per feature, one column per sample,

plus whatever annotation columns the report carries. The peptide type is different — it is the long report, one row per measurement, and the tool stores only the path to it rather than loading it into the session, so it has no preview and no row count on this page.

> **WORTH KNOWING**
>
> Nothing about the upload is level-specific except which of the four slots the table lands in. A phosphosite report uploaded as **Protein** is accepted; it will simply be described as protein data everywhere afterwards.

What the tool does to the table

Four things happen to every wide table between the file and the session, in this order, and all four are visible in the preview underneath:

1. Column names are stripped of surrounding whitespace.
2. Eight column families are renamed to the canonical names the rest of the tool uses (below). A name that is already canonical, or whose target is already present in the table, is left alone.
3. Repeated column names are made unique by appending .1, .2 and so on, so that two columns called Genes do not silently become one.
4. Every column that is not numeric is stored as text.

2 Columns the tool looks for

The report does not have to use these names — the aliases on the right are recognised and renamed on the way in. The name on the left is what appears in the preview and in every later table.

Columns renamed on upload

Column	What it holds
File.Name	The run. Recognised from Run or R.FileName.
ProteinNames	The protein group. Recognised from PG.ProteinNames, PG.ProteinGroups, ProteinIds or ProteinNames.
GeneNames	The gene. Recognised from Genes, GeneNames or PG.Genes.
Stripped.Sequence	The unmodified peptide sequence. Recognised from PEP.StrippedSequence.
Modified.Sequence	The modified peptide sequence. Recognised from EG.ModifiedSequence.
Precursor.Quantity	The precursor quantity. Recognised from FG.Quantity.
RT	Retention time. Recognised from EG.ApexRT.
Predicted.RT	Predicted retention time. Recognised from EG.RTPredicted.

Matching ignores case, spaces and punctuation, so PG.Protein Names, pg_proteinnames and PG.ProteinNames are all the same name. Any other column is kept exactly as it is written.

Columns a phospho file needs for collapsing

A phosphosite table can be uploaded without them, but it cannot be collapsed to phosphoproteins unless it carries all five of UPD_seq, PTM_localization, Protein_group, Gene_group and PTM_Collapse_key, and unless each Protein_group maps to exactly one Gene_group. The collapse refuses the table by name when one is missing.

Columns a peptide report needs

The peptide pages look for a run column — File.Name, or the first column whose name contains *file* — a stripped-sequence column, and Precursor.Quantity for anything that pivots a quantity. A missing one is reported when the page that needs it is opened, not on upload, because the file is never read here.

3 Uploading

1. Choose the level in **Dataset type**. The choice decides which of the four slots the table fills, and nothing else.
2. Choose the file. The browser's file dialogue is restricted to the five accepted extensions.
3. Press **Upload**. A wide table is parsed immediately, so a file that cannot be read is refused now rather than three pages later.
4. Check the preview: the row and column counts, and whether the sample columns are the ones you expect and carry numbers rather than text.
5. Go to Data → Annotation and assign the sample columns to conditions. Until that is done the tool knows the table but not the experiment.

When the browser exposes the file's own location on disk, that path is stored and the file is read from where it is. Otherwise a copy is written to a working directory and its path is stored instead. Either way the path is shown as Restore Path on the **Current datasets** card, and it is what a saved analysis is restored from.

> EASY TO GET WRONG
>
> A file that uploads cleanly has not been understood. Upload decides nothing about which columns are samples — the annotation does, and it is where a mistyped or unselected column becomes a missing replicate. Two things are worth checking in

> the preview before moving on: that every run you expect has a column, and that its cells are numbers. A quantity column exported with a decimal comma parses as text, and every value in it is read as not detected; the annotation reports it as a warning, but only once the column has been assigned to a condition.

When a file is refused

The parser's own wording is replaced by the cause wherever the cause is known. The refusals that occur in practice:

- **Unsupported file type.** The extension is not one of the five.
- **A NUL byte where text was expected.** Almost always a file saved as UTF-16, or truncated mid-write. Re-save it as UTF-8 text; it is refused rather than repaired because the C parser would otherwise truncate the header at that byte and rename a column silently.
- **Not valid UTF-8.** Re-export, or re-save, as UTF-8.
- **Rows do not all have the same number of columns.** A ragged export, or a stray delimiter inside an unquoted field.
- **No columns to parse.** The file is empty, or its first line is not the header.
- **Named** .xlsx **but not a valid workbook.** Usually a .xls or a CSV renamed by hand.
- **No data rows.** The table parsed, and the header is all there is.
- **Only one column.** A report needs an identifier column and at least one measurement column. This is what a saved web page, or the wrong separator, looks like from here.

The default upload ceiling is 64 GiB, with at most 20,000 columns and 10 billion cells. A separate parsed-table memory guard normally uses a quarter of detected physical/container memory, capped at 128 GiB; it falls back to 1 GiB when capacity cannot be detected. Administrators can change these limits, and a hosting proxy may impose a smaller upload limit.

These are safety ceilings, not a promise that a file of that size can be analysed on every computer. Compressed Excel and Parquet files can expand considerably. Parsing and later analyses also need working memory beyond the stored table. For large reports, prefer a suitable tabular export and enough server RAM; raising a limit does not make the analysis out-of-core.

4 Controls

Controls on this page

Control	What it does	Default	Where
Dataset type	Which level the file is: Protein, Phospho, Phosphoprotein or Peptide. Also selects which dataset the preview below describes.	Protein	page
File	The file to upload. Restricted to the five accepted extensions.	none	page
Upload	Reads and stores the file. Disabled until a file is chosen.	—	page
Show Dataset Manager	Reveals the three controls below. The choice is remembered between visits.	hidden	page
Selected dataset	Which dataset package is active. A package holds one slot per level, so a second experiment can be loaded beside the first without overwriting it.	Data	page
New dataset	Creates an empty package under the name given and makes it active.	empty	page
Rename selected dataset	Renames the active package. The annotation and metadata profiles attached to it follow the name.	the active package	page

Every plot, table and test in the session reads the active package. The level selector on a plot page offers only the levels that package actually holds.

> NOTE
>
> In viewer mode the upload form and the dataset manager are not drawn at all. What remains is a **Dataset level preview** selector over the datasets the instance was configured with.

5 What the upload replaces

Uploading a protein, phospho or phosphoprotein file into a slot that already holds one replaces it, and discards two things that were derived from the old table: the annotation for that level, in every metadata profile, and — for protein and phospho — the uploaded metadata for that level. Both are discarded rather than re-applied because a metadata file matches samples by column name, and the new table's columns may be different ones. Uploading a peptide file clears the peptide metadata of the active package in the same way.

Nothing else is touched: the other three levels, the other packages and the condition palette all survive.

6 Example data

The Python package ships a real HEK293 experiment under proteomicscopylot/example_data: ten samples, five wild type and five GRK, run as both a whole-proteome and a phosphoproteome arm. It is what example.ipynb and example.R run on, and the files upload here like any others — a protein-group report of 7,490 rows, a phosphosite report of 11,916 rows, a phosphoprotein matrix of 2,301 rows, and a sample-to-condition table for each arm that can be uploaded as metadata on the annotation page. The protein report carries PG.Log2Quantity, so its quantities are already on the $\log_2$ scale and the annotation detects that without being told. In a notebook, load_example_data("protein") reads it, and the same function takes "phospho", "phosphoprotein", "protein_metadata" and "phospho_metadata", in Python and in R.

There is no button that loads them: the example files are uploaded on this page like any other dataset, which is deliberate — trying the tool on them is the same procedure as using it on your own data, so nothing learnt on the example has to be unlearnt afterwards.

7 Exporting

Nothing is exported from this page. The table as uploaded is downloadable once it has been annotated, from Data → Annotation, as <level>_data_full.csv.

Behind this page

Function	What it produces
read_dataframe	Reads an uploaded table and refuses one that is not a table
get_extension	Accepts or refuses the file type by name
rename_cols	Renames the eight recognised column families to their canonical names
make_columns_unique	Deduplicates repeated column names
sanitize_dataframe	Stores every non-numeric column as text
save_table_dataset	Puts the parsed table in the active package
save_peptide_path	Stores the peptide report's path rather than its contents